

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт Перспективной инженерии
Департамент цифровых и робототехнических систем и электроники

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 6
дисциплины «Искусственный Интеллект и Машинное Обучение»

Выполнил: Мендеш Пашкоал Педру
2 курс, группа ИВТ-6-о-24-1,
09.03.01 «Информатика и Вычислительная
техника», направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная форма
обучения

(подпись)

Руководитель практики:
Воронкин Роман А. доцент факультета
цифровых, робототехнических систем и
электроники института перспективной
инженерии.

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: Введение в pandas: изучение структуры Series и базовых операций

Цель работы: познакомить с основами работы с библиотекой pandas, в частности, со структурой данных Series.

Этапы выполнения лабораторных работ

1. Изучение теоретического материала
2. Создание репозитория на GitHub:

<https://github.com/Pascoalpm/eda-pandas-lab.git>

Comentado [PM1]: <https://github.com/Pascoalpm/eda-pandas-lab.git>

3. Клонирование репозитория:
4. Настройка .gitignore для Python, Jupyter.
5. Запуск JupyterLab:
6. Выполнение заданий

Примеры практической работы

ПРИМЕР 1: Обнаружение пропущенных значений (.isna(), .notna())

```
import pandas as pd
import numpy as np

data = {
    "Имя": ["Анна", "Иван", "Ольга", None, "Мария"],
    "Возраст": [25, 30, None, 40, 35],
    "Город": ["Москва", None, "Казань", "Новосибирск", "СПб"]
}

df = pd.DataFrame(data)
print("DataFrame:")
print(df)

print("\n.isna() - проверка на пропуски:")
print(df.isna())

print("\n.isna().sum() - количество пропусков по столбцам:")
print(df.isna().sum())
```

DataFrame:

	Имя	Возраст	Город
0	Анна	25.0	Москва
1	Иван	30.0	NaN
2	Ольга	NaN	Казань
3	NaN	40.0	Новосибирск
4	Мария	35.0	СПб

.isna() - проверка на пропуски:

	Имя	Возраст	Город
0	False	False	False
1	False	False	True
2	False	True	False
3	True	False	False
4	False	False	False

.isna().sum() - количество пропусков по столбцам:

	Имя	Возраст	Город
	1	1	1

dtype: int64

Рисунок 1

ПРИМЕР 2: Визуализация пропусков с missingno

```
import pandas as pd
import numpy as np
import missingno as mso
import matplotlib.pyplot as plt

np.random.seed(42)

data = {
    "Имя": ["Анна", "Маша", "Ольга", "Петр", "Мария", "Дмитрий", "Елена", "Сергей", "Алина", "Артем"],
    "Возраст": [25, np.nan, 22, 48, 35, np.nan, 28, 31, np.nan, 29],
    "Город": ["Москва", "СПб", np.nan, "Новосибирск", "СПб", "Екатеринбург", np.nan, "Казань", "Томск", np.nan],
    "Доход": [50000, 60000, np.nan, 70000, 65000, 55000, 40000, np.nan, 52000, 58000],
    "Образование": [np.nan, "Высшее", "Среднее", "Высшее", np.nan, "Высшее", "Среднее", np.nan, "Среднее"],
    "Стаж работы": [3, 8, 1, 15, 10, np.nan, 5, 7, np.nan, 2]
}

df = pd.DataFrame(data)

print("DataFrame:")
print(df)

print("\n" + "-"*50)
print("Матрица пропусков (mso.matrix()):")
print("-"*50)
mso.matrix(df)
plt.title("Матрица пропусков")
plt.show()

print("Bar-график пропусков (mso.bar()):")
mso.bar(df)
plt.title("Bar-график пропусков по столбцам")
plt.show()

DataFrame:
   Имя  Возраст  Город  Доход  Образование  Стаж работы
0  Анна    25.0  Москва  50000.0         NaN          3.0
1  Маша    NaN   СПб    60000.0         Высшее          8.0
2  Ольга    22.0    NaN         NaN         Среднее          1.0
3  Петр    48.0  Новосибирск  70000.0         Высшее          15.0
4  Мария    35.0   СПб    65000.0         NaN          10.0
5  Дмитрий  NaN  Екатеринбург  55000.0         Высшее         NaN
6  Елена    28.0    NaN    40000.0         Среднее          5.0
7  Сергей    31.0  Казань         NaN         Высшее          7.0
8  Алина    NaN   Томск    52000.0         NaN         NaN
9  Артем    29.0    NaN    58000.0         Среднее          2.0
```

Рисунок 2

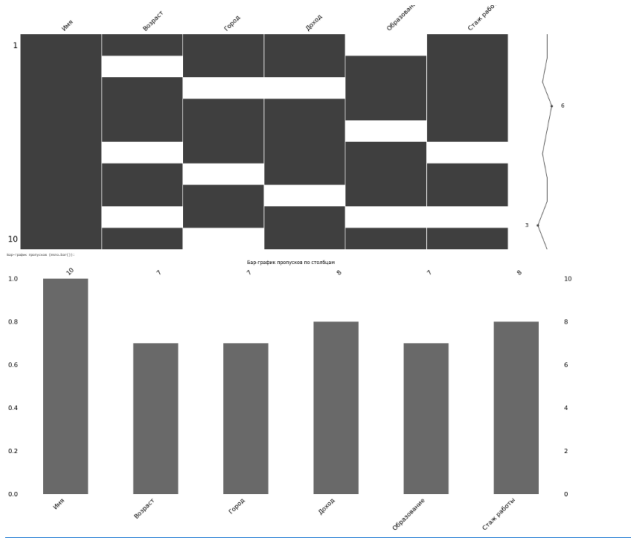


Рисунок 3

ПРИМЕР 3: Удаление пропущенных значений (.dropna())

```
data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария", "Дмитрий"],
    "Возраст": [25, пр.пан, 22, 40, 35, пр.пан],
    "Город": ["Москва", "СПб", пр.пан, "Новосибирск", "СПб", "Екатеринбург"],
    "Доход": [50000, 60000, пр.пан, 70000, 65000, 55000]
}

df = pd.DataFrame(data)
print("Исходный DataFrame:")
print(df)

print("\n.dropna() - удаление строк с любым NaN:")
print(df.dropna())

print("\n.dropna(axis=1) - удаление столбцов с любым NaN:")
print(df.dropna(axis=1))

print("\n.dropna(thresh=3) - удаление строк, где менее 3 непустых значений:")
print(df.dropna(thresh=3))
```

Исходный DataFrame:

	Имя	Возраст	Город	Доход
0	Анна	25.0	Москва	50000.0
1	Иван	NaN	СПб	60000.0
2	Ольга	22.0	NaN	NaN
3	Петр	40.0	Новосибирск	70000.0
4	Мария	35.0	СПб	65000.0
5	Дмитрий	NaN	Екатеринбург	55000.0

```
.dropna() - удаление строк с любым NaN:
Имя  Возраст  Город  Доход
0  Анна    25.0  Москва  50000.0
3  Петр    40.0  Новосибирск  70000.0
4  Мария   35.0    СПб    65000.0

.dropna(axis=1) - удаление столбцов с любым NaN:
Имя
0  Анна
1  Иван
2  Ольга
3  Петр
4  Мария
5  Дмитрий

.dropna(thresh=3) - удаление строк, где менее 3 непустых значений:
Имя  Возраст  Город  Доход
0  Анна    25.0  Москва  50000.0
1  Иван    NaN    СПб    60000.0
3  Петр    40.0  Новосибирск  70000.0
4  Мария   35.0    СПб    65000.0
5  Дмитрий  NaN  Екатеринбург  55000.0
```

Рисунок 4

ПРИМЕР 4: Заполнение пропусков (.fillna())

```
data = {
    "Имя": ["Анна", "Иван", "Ольга", None, "Мария"],
    "Возраст": [25, 30, None, 40, 35],
    "Город": ["Москва", None, "Казань", "Новосибирск", "СПб"]
}

df = pd.DataFrame(data)
print("Исходный DataFrame:")
print(df)

print("\n.fillna(30) - заполнение константой:")
df_const = df.copy()
df_const["Возраст"] = df_const["Возраст"].fillna(30)
print(df_const)

print("\n.fillna(mean) - заполнение средним значением:")
df_mean = df.copy()
mean_age = df_mean["Возраст"].mean()
df_mean["Возраст"] = df_mean["Возраст"].fillna(mean_age)
print(df_mean)

print("\n.fillna(method='ffill') - заполнение предыдущим значением:")
df_ffill = df.copy()
df_ffill["Возраст"] = df_ffill["Возраст"].fillna(method="ffill")
print(df_ffill)

print("\n.fillna(method='bfill') - заполнение следующим значением:")
df_bfill = df.copy()
df_bfill["Возраст"] = df_bfill["Возраст"].fillna(method="bfill")
print(df_bfill)
```

Исходный DataFrame:

	Имя	Возраст	Город
0	Анна	25.0	Москва
1	Иван	30.0	NaN
2	Ольга	NaN	Казань
3	NaN	40.0	Новосибирск
4	Мария	35.0	СПб

```
.fillna(30) - заполнение константой:
Имя  Возраст  Город
0  Анна    25.0  Москва
1  Иван    30.0    NaN
2  Ольга    30.0  Казань
3  NaN     40.0  Новосибирск
4  Мария   35.0    СПб

.fillna(mean) - заполнение средним значением:
Имя  Возраст  Город
0  Анна    25.0  Москва
1  Иван    30.0    NaN
2  Ольга   32.5  Казань
3  NaN     40.0  Новосибирск
4  Мария   35.0    СПб
```

Рисунок 5

ПРИМЕР 5: Интерполяция (.interpolate())

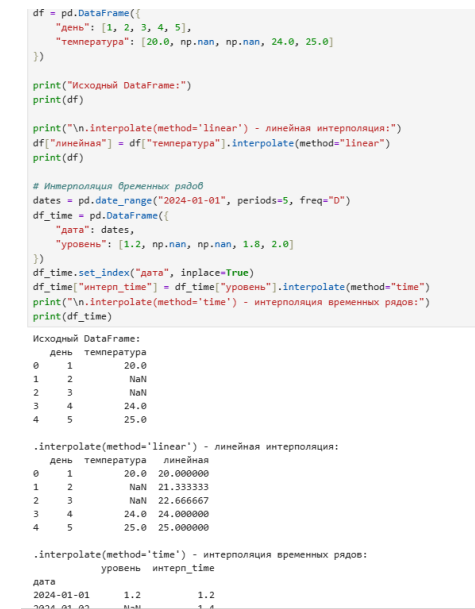


Рисунок 6

ПРИМЕР 6: Обнаружение выбросов (метод IQR)



Рисунок 7

ПРИМЕР 7: Визуализация выбросов (Boxplot)

```
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария", "Дмитрий", "Елена"],
    "Возраст": [25, 30, 22, 40, 35, 85, 5]
})

plt.figure(figsize=(6, 4))
sns.boxplot(x=df["Возраст"])
plt.title("Boxplot: Возраст")
plt.xlabel("Возраст")
plt.grid(True)
plt.show()
```

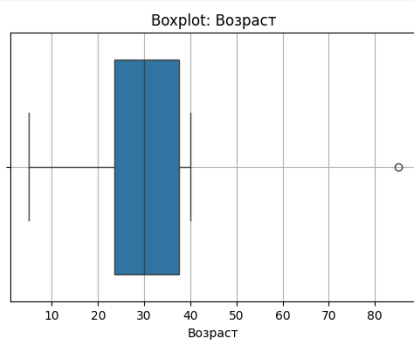


Рисунок 8

ПРИМЕР 8: Стандартизация признаков (StandardScaler)

```
from sklearn.preprocessing import StandardScaler
import pandas as pd

df = pd.DataFrame({
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Возраст": [25, 30, 22, 40, 35],
    "Зарплата": [50000, 60000, 45000, 70000, 65000]
})

print("Исходный DataFrame:")
print(df)

scaler = StandardScaler()
df_scaled = df.copy()
df_scaled[["Возраст", "Зарплата"]] = scaler.fit_transform(df[["Возраст", "Зарплата"]])

print("\nDataFrame после стандартизации (среднее=0, std=1):")
print(df_scaled)

print(f"\nСреднее возраста: {df_scaled['Возраст'].mean():.2f}")
print(f"Стандартное отклонение возраста: {df_scaled['Возраст'].std():.2f}")

Исходный DataFrame:
   Имя  Возраст  Зарплата
0  Анна      25    50000
1  Иван      30    60000
2  Ольга      22    45000
3  Петр       40    70000
4  Мария      35    65000

DataFrame после стандартизации (среднее=0, std=1):
   Имя  Возраст  Зарплата
0  Анна -0.826961 -0.862662
1  Иван -0.061256  0.215666
2  Ольга -1.286384 -1.401826
3  Петр  1.470153  1.293993
4  Мария  0.704448  0.754829

Среднее возраста: 0.00
Стандартное отклонение возраста: 1.12
```

Рисунок 9

ПРИМЕР 9: Нормализация признаков (MinMaxScaler)

```
from sklearn.preprocessing import MinMaxScaler

df = pd.DataFrame({
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Возраст": [25, 30, 22, 40, 35],
    "Зарплата": [50000, 60000, 45000, 70000, 65000]
})

print("Исходный DataFrame:")
print(df)

scaler = MinMaxScaler()
df_norm = df.copy()
df_norm[["Возраст", "Зарплата"]] = scaler.fit_transform(df[["Возраст", "Зарплата"]])

print("\nDataFrame после нормализации (диапазон [0, 1]):")
print(df_norm)

print(f"\nМинимальное возраста: {df_norm['Возраст'].min():.2f}")
print(f"Максимальное возраста: {df_norm['Возраст'].max():.2f}")
```

Исходный DataFrame:

	Имя	Возраст	Зарплата
0	Анна	25	50000
1	Иван	30	60000
2	Ольга	22	45000
3	Петр	40	70000
4	Мария	35	65000

DataFrame после нормализации (диапазон [0, 1]):

	Имя	Возраст	Зарплата
0	Анна	0.166667	0.2
1	Иван	0.444444	0.6
2	Ольга	0.000000	0.0
3	Петр	1.000000	1.0
4	Мария	0.722222	0.8

Минимальное возраста: 0.00
Максимальное возраста: 1.00

Рисунок 10

ПРИМЕР 10: Label Encoding с pandas.map()

```
df = pd.DataFrame({
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Образование": ["среднее", "высшее", "начальное", "высшее", "среднее"]
})

print("Исходный DataFrame:")
print(df)

education_order = {
    "начальное": 0,
    "среднее": 1,
    "высшее": 2
}

df["Образование_encoded"] = df["Образование"].map(education_order)
print("\nDataFrame после Label Encoding (с заданным порядком):")
print(df)
```

Исходный DataFrame:

	Имя	Образование
0	Анна	среднее
1	Иван	высшее
2	Ольга	начальное
3	Петр	высшее
4	Мария	среднее

DataFrame после Label Encoding (с заданным порядком):

	Имя	Образование	Образование_encoded
0	Анна	среднее	1
1	Иван	высшее	2
2	Ольга	начальное	0
3	Петр	высшее	2
4	Мария	среднее	1

Рисунок 11

ПРИМЕР 11: One-Hot Encoding с pandas.get_dummies()

```
df = pd.DataFrame({
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Город": ["Москва", "СПб", "Казань", "Москва", "Казань"]
})

print("Исходный DataFrame:")
print(df)

df_encoded = pd.get_dummies(df, columns=["Город"], drop_first=True)
print("\nDataFrame после One-Hot Encoding (drop_first=True):")
print(df_encoded)
```

Исходный DataFrame:

	Имя	Город
0	Анна	Москва
1	Иван	СПб
2	Ольга	Казань
3	Петр	Москва
4	Мария	Казань

DataFrame после One-Hot Encoding (drop_first=True):

	Имя	Город_Москва	Город_СПб
0	Анна	True	False
1	Иван	False	True
2	Ольга	False	False
3	Петр	True	False
4	Мария	False	False

Рисунок 12

ПРИМЕР 12: Комплексный EDA на датасете Diamonds

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import missingno as msno
from sklearn.preprocessing import StandardScaler

print("-"*60)
print("КОМПЛЕКСНЫЙ EDA: ДАТАСЕТ DIAMONDS")
print("-"*60)

# 1. Загрузка данных
print("\n1. ЗАГРУЗКА ДАННЫХ:")
df = pd.read_csv("https://raw.githubusercontent.com/Pratik-Bhujade/Diamond-Dataset/refs/heads/master/DiamondData.csv")
print(df.head())

# 2. Удаление целевого признака
df.drop(columns=["price"], inplace=True)

# 3. Информация о данных
print("\n2. ИНФОРМАЦИЯ О ДАННЫХ:")
df.info()
print(df.describe())

# 4. Обнаружение пропусков
print("\n3. ОБНАРУЖЕНИЕ ПРОПУСКОВ:")
print(df.isna().sum())

# 5. Визуализация пропусков
print("\n4. ВИЗУАЛИЗАЦИЯ ПРОПУСКОВ:")
msno.matrix(df)
plt.title("Матрица пропусков - Diamonds")
plt.show()

msno.bar(df)
plt.title("Бар-график пропусков - Diamonds")
plt.show()

# 6. Удаление пропусков
print("\n5. УДАЛЕНИЕ ПРОПУСКОВ:")
df = df.dropna()
```

Рисунок 13

Задание 1: Обнаружение и обработка пропущенных значений

```
# =====
# ЗАДАНИЕ 1: ОБНАРУЖЕНИЕ И ОБРАБОТКА ПРОПУЩЕННЫХ ЗНАЧЕНИЙ
# DATASET: TITANIC (OFFLINE - ИСПОЛНЕННЫЙ ВЕРСИОМ)
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import missingno as msno

print("\n40")
print("ЗАДАНИЕ 1: ОБНАРУЖЕНИЕ И ОБРАБОТКА ПРОПУЩЕННЫХ ЗНАЧЕНИЙ")
print("DATASET: TITANIC (OFFLINE)")
print("\n40")

# 1. СОЗДАНИЕ ДАННЫХ ВРУЧНУЮ
print("\n1. СОЗДАНИЕ ДАННЫХ:")

n = 801
np.random.seed(42)

# Функция для создания списка с профильными вероятностями
def make_choice(values, probs, n):
    # Гарантируем, что сумма вероятностей = 1
    probs = np.array(probs) / np.sum(probs)
    return np.random.choice(values, n, p=probs)

# Создание DataFrame
df = pd.DataFrame({
    "passenger_id": range(1, n + 1),
    "survived": np.random.choice([0, 1], n, p=[0.62, 0.38]),
    "pclass": np.random.choice([1, 2, 3], n, p=[0.24, 0.21, 0.55]),
    "sex": np.random.choice(["male", "female"], n, p=[0.65, 0.35]),
    "age": np.random.normal(30, 14, n),
    " SibSp": make_choice([0, 1, 2, 3, 4, 5], [0.68, 0.23, 0.04, 0.03, 0.01, 0.01], n),
    "parch": make_choice([0, 1, 2, 3, 4, 5], [0.76, 0.13, 0.04, 0.02, 0.01, 0.04], n),
    "fare": np.random.normal(52, 50, n),
    "embarked": np.random.choice(["C", "Q", "S"], n, p=[0.19, 0.09, 0.72]),
    "embark_town": np.random.choice(["Cherbourg", "Queenstown", "Southampton"], n, p=[0.19, 0.09, 0.72])
})

# Добавление имен
names = ["Brand, Mr. Owen Harris", "Cotings, Mrs. John Bradley", "Heikkinen, Miss. Laina",
         "Futrelle, Mrs. Jacques Heath", "Allen, Mr. William Henry", "Moran, Mr. James",
         "McCarthy, Mr. Timothy J", "Palsson, Master. Gosta Leonard", "Johnson, Mrs. Oscar W",
         "Nasser, Mr. Nicholas", "Sandstrom, Miss. Marguerite Rut", "Monnell, Miss. Elizabeth",
         "Saunderscock, Mr. William Henry", "Andersson, Mr. Anders Johan", "Vestrom, Miss. Hulda Amanda",
         "Headett, Mrs. Mary D King", "Rice, Master. Eugene", "Williams, Mr. Charles Eugene",
         "Vander Planke, Mrs. Julius", "Wassellani, Mrs. Fatima"]
df["name"] = np.random.choice(names, n)

print("Первые 5 строк:")
print(df.head())
print(f"Внешний dataset: {df.shape}")

# 2. ДОБАВЛЕНИЕ ПРОПУСКОВ (NaN)
print("\n2. ДОБАВЛЕНИЕ ПРОПУСКОВ:")

# age - 20% пропущено
age_nan_idx = np.random.choice(df.index, size=int(0.2 * n), replace=False)
df.loc[age_nan_idx, "age"] = np.nan

# embarked - 10% пропущено
embarked_nan_idx = np.random.choice(df.index, size=int(0.1 * n), replace=False)
df.loc[embarked_nan_idx, "embarked"] = np.nan

# embark_town - 8% пропущено
town_nan_idx = np.random.choice(df.index, size=int(0.08 * n), replace=False)
df.loc[town_nan_idx, "embark_town"] = np.nan

# fare - 5% пропущено
fare_nan_idx = np.random.choice(df.index, size=int(0.05 * n), replace=False)
df.loc[fare_nan_idx, "fare"] = np.nan

print(f"Пропущено в age: {df['age'].isna().sum()}")
print(f"Пропущено в embarked: {df['embarked'].isna().sum()}")
print(f"Пропущено в embark_town: {df['embark_town'].isna().sum()}")
print(f"Пропущено в fare: {df['fare'].isna().sum()}")

# 3. ИНФОРМАЦИЯ О ДАННЫХ
print("\n3. ИНФОРМАЦИЯ О ДАННЫХ:")
df.info()

print("\n4. СТАТИСТИЧЕСКОЕ ОПИСАНИЕ:")
print(df.describe())

# 4. ОБНАРУЖЕНИЕ ПРОПУСКОВ
print("\n5. ОБНАРУЖЕНИЕ ПРОПУСКОВ:")
print("Количество NaN по столбцам:")
print(df.isna().sum())

print("\nПроцент NaN по столбцам:")
print((df.isna().sum() / len(df)) * 100)

# 5. ВИЗУАЛИЗАЦИЯ ПРОПУСКОВ
print("\n6. ВИЗУАЛИЗАЦИЯ ПРОПУСКОВ:")

# Matrix
plt.figure(figsize=(12, 6))
msno.matrix(df)
plt.title("Матрица пропусков - Titanic")
plt.xlabel("Индекс строки")
plt.ylabel("Столбцы")
plt.show()

# Bar
plt.figure(figsize=(10, 5))
msno.bar(df)
plt.title("Бар-график пропусков - Titanic")
plt.xlabel("Столбцы")
plt.ylabel("Количество заполненных значений")
plt.show()
```

Рисунок 14

```

# Heatmap
plt.figure(figsize=(8, 6))
msno.heatmap(df)
plt.title("Тепловая карта пропусков - Titanic")
plt.show()

# 6. ОБРАБОТКА ПРОПУСКОВ
print("\n7. ОБРАБОТКА ПРОПУСКОВ:")

df_clean = df.copy()

# 6.1 Age - заполняем средним значением
mean_age = df_clean["age"].mean()
df_clean["age"] = df_clean["age"].fillna(mean_age)
print(f"- age: заполнено средним ({mean_age:.2f})")

# 6.2 Embarked - заполняем модой
mode_embarked = df_clean["embarked"].mode()[0]
df_clean["embarked"] = df_clean["embarked"].fillna(mode_embarked)
print(f"- embarked: заполнено модой ({mode_embarked})")

# 6.3 Embark_town - заполняем модой
mode_embark_town = df_clean["embark_town"].mode()[0]
df_clean["embark_town"] = df_clean["embark_town"].fillna(mode_embark_town)
print(f"- embark_town: заполнено модой ({mode_embark_town})")

# 6.4 Fare - заполняем медианой
median_fare = df_clean["fare"].median()
df_clean["fare"] = df_clean["fare"].fillna(median_fare)
print(f"- fare: заполнено медианой ({median_fare:.2f})")

# 7. ПРОВЕРКА РЕЗУЛЬТАТА
print("\n8. ПРОВЕРКА РЕЗУЛЬТАТА:")
print("Число после обработки:")
print(df_clean.isna().sum())

print("\nИнформация после обработки:")
df_clean.info()

# 8. СРАВНЕНИЕ ДД И ПОСЛЕ
print("\n9. СРАВНЕНИЕ ДД И ПОСЛЕ ОБРАБОТКИ:")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

msno.matrix(df, ax=axes[0])
axes[0].set_title("ДД обработки пропусков")

msno.matrix(df_clean, ax=axes[1])
axes[1].set_title("ПОСЛЕ обработки пропусков")

plt.tight_layout()
plt.show()

# 10. СРАВНЕНИЕ СТАТИСТИК
print("\n10. СРАВНЕНИЕ СТАТИСТИК (age):")
print(f"DD: среднее={df['age'].mean():.2f}, медиана={df['age'].median():.2f}")
print(f"ПОСЛЕ: среднее={df_clean['age'].mean():.2f}, медиана={df_clean['age'].median():.2f}")

print("\n" + "="*60)
print("10. ЗАДАНИЕ 1 ВЫПОЛНЕНО!")
print("\n" + "="*60)

=====
ЗАДАНИЕ 1: ОБНАРУЖЕНИЕ И ОБРАБОТКА ПРОПУЩЕННЫХ ЗНАЧЕНИЙ
ДАТАСЕТ: TITANIC (OFFLINE)
=====

1. СОЗДАНИЕ ДАННЫХ:
Первые 5 строк:
   passenger_id  survived  pclass  sex    age  sibsp  parch    fare \
0             1         0       3   male  32.692182    1    1  89.432719
1             2         1       3   male  27.924888    1    0 -17.564519
2             3         1       2  female  17.701248    0    0  78.029912
3             4         0       3  female  10.224699    0    0  -6.071488
4             5         0       1   male   5.586126    0    0  15.275387

   embarked  embark_town  name
0          S  Southampton  Vestrom, Miss. Hulda Amanda
1          S  Southampton  Futrelle, Mrs. Jacques Heath
2          S   Cherbourg  Saendercock, Mr. William Henry
3          S  Southampton  Bonnell, Miss. Elizabeth
4          S  Queenstown  Allen, Mr. William Henry

Размер dataset: (891, 11)

2. ДОБАВЛЕНИЕ ПРОПУСКОВ:
Пропусков в age: 178
Пропусков в embarked: 89
Пропусков в embark_town: 71
Пропусков в fare: 44

3. ИНФОРМАЦИЯ О ДАННЫХ:
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0   passenger_id  891 non-null    int64
1   survived      891 non-null    int64
2   pclass        891 non-null    int64
3   sex           891 non-null    str
4   age           713 non-null    float64
5   sibsp         891 non-null    int64
6   parch        891 non-null    int64
7   fare         847 non-null    float64
8   embarked     802 non-null    str
9   embark_town  820 non-null    str
10  name          891 non-null    str
dtypes: float64(2), int64(5), str(4)
memory usage: 76.7 KB

4. СТАТИСТИЧЕСКОЕ ОПИСАНИЕ:
   passenger_id  survived  pclass    age    sibsp \
count    891.000000    891.000000    891.000000    713.000000    891.000000
mean     446.000000    0.373737    2.308642    29.129725    0.442200
std      257.353842    0.484067    0.845426    13.802432    0.871376
min        1.000000    0.000000    1.000000   -18.500018    0.000000
25%     223.500000    0.000000    1.500000    19.758012    0.000000
50%     446.000000    0.000000    2.000000    29.272725    0.000000

```

Рисунок 15

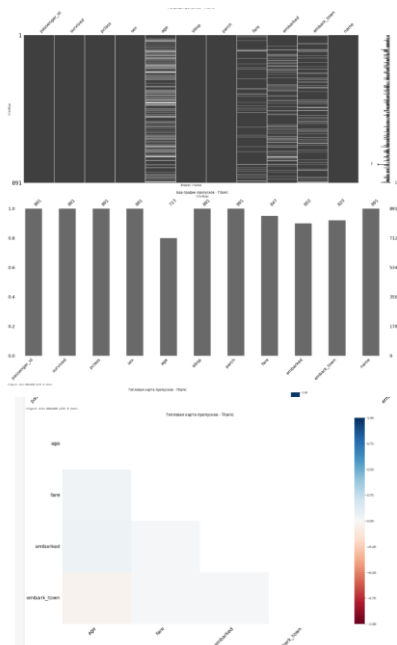


Рисунок 16

ЗАДАНИЕ 2: Обнаружение и удаление выбросов (Penguins)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("\n")
print("ЗАДАНИЕ 2: ОБНАРУЖЕНИЕ И УДАЛЕНИЕ ВЫБРОСОВ")
print("ДАТАСЕТ: PENGUINS (OFFLINE)")
print("\n")

# 1. СОЗДАНИЕ ДАННЫХ
print("\n1. СОЗДАНИЕ ДАННЫХ:")

np.random.seed(42)
n = 200

df = pd.DataFrame({
    "species": np.random.choice(["Adelie", "Chinstrap", "Gentoo"], n),
    "bill_length_mm": np.random.normal(44, 6, n),
    "bill_depth_mm": np.random.normal(17, 3, n),
    "flipper_length_mm": np.random.normal(180, 14, n),
    "body_mass_g": np.random.normal(4200, 800, n),
    "sex": np.random.choice(["Male", "Female"], n)
})

# Добавление outliers
df.loc[10, "bill_length_mm"] = 80
df.loc[20, "bill_depth_mm"] = 35
df.loc[30, "flipper_length_mm"] = 300
df.loc[40, "body_mass_g"] = 8000

print("Первые 5 строк:")
print(df.head())
print(f"Shape: {df.shape}")

# 2. Визуализация выбросов
print("\n2. ВИЗУАЛИЗАЦИЯ ВЫБРОСОВ:")

numeric_cols = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

for i, col in enumerate(numeric_cols):
    sns.boxplot(x=df[col], y=axes[i])
    axes[i].set_title(f"Boxplot: {col}")
    axes[i].grid(True)

plt.tight_layout()
plt.show()

# 3. УДАЛЕНИЕ ВЫБРОСОВ (ZDR)
print("\n3. УДАЛЕНИЕ ВЫБРОСОВ (МЕТОД ZDR):")
```

Рисунок 17

```
def remove_outliers_iqr(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    return data[(data[column] >= lower) & (data[column] <= upper)]

print(f"Размер до удаления: {len(df)}")

df_clean = df.copy()
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower) | (df[col] > upper)]
    print(f"col: {col}, Q1={Q1:.2f}, Q3={Q3:.2f}, IQR={IQR:.2f}, выбросов={len(outliers)}")
    df_clean = remove_outliers_iqr(df_clean, col)

print(f"Размер после удаления: {len(df_clean)}")
print(f"Удалено: {len(df) - len(df_clean)} строк")

# 4. Визуализация после
print("\n4. Визуализация после удаления:")

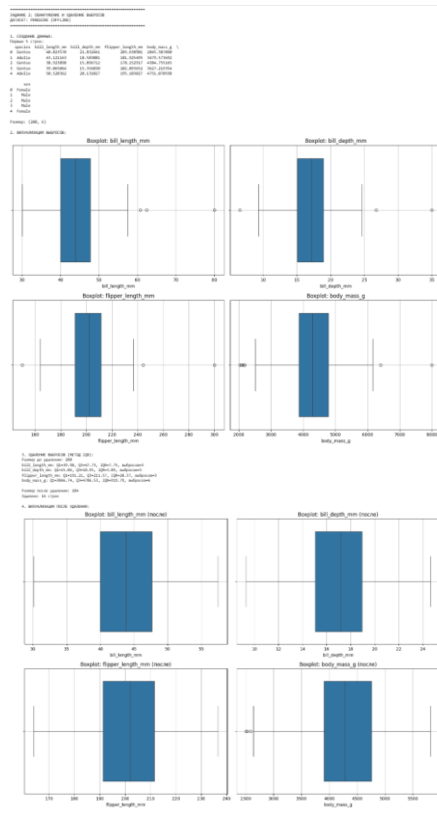
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

for i, col in enumerate(numeric_cols):
    sns.boxplot(x=df_clean[col], ax=axes[i])
    axes[i].set_title(f"Boxplot: {col} (после)")
    axes[i].grid(True)

plt.tight_layout()
plt.show()

print("\nЗадача 2 выполнена!")
```

Рисунок 18



ЗАДАНИЕ 3: Масштабирование числовых признаков

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler, MinMaxScaler

print("-"*60)
print("ЗАДАНИЕ 3: МАСШТАБИРОВАНИЕ ЧИСЛОВЫХ ПРИЗНАКОВ")
print("DATASET: CALIFORNIA HOUSING (OFFLINE)")
print("-"*60)

# 1. СОЗДАНИЕ ДАННЫХ
print("\n1. СОЗДАНИЕ ДАННЫХ:")

np.random.seed(42)
n = 1000

df = pd.DataFrame({
    "MedInc": np.random.normal(5, 3, n),
    "HouseAge": np.random.randint(1, 50, n),
    "AveRooms": np.random.normal(5, 2, n),
    "AveBedrms": np.random.normal(1, 0.5, n),
    "Population": np.random.randint(100, 5000, n),
    "AveOccup": np.random.normal(2, 0.5, n),
    "Latitude": np.random.normal(35, 5, n),
    "Longitude": np.random.normal(-120, 5, n)
})

print("Первые 5 строк:")
print(df.head())
print(f"Вспомог: {df.shape}")
print(f"Признаки: {df.columns.tolist()}")

# 2. СТАНДАРТИЗАЦИЯ
print("\n2. СТАНДАРТИЗАЦИЯ (StandardScaler):")

scaler_std = StandardScaler()
df_std = df.copy()
df_std[df.columns] = scaler_std.fit_transform(df[df.columns])

print("После стандартизации (среднее=0, std=1):")
print(df_std.describe())

# 3. НОРМАЛИЗАЦИЯ
print("\n3. НОРМАЛИЗАЦИЯ (MinMaxScaler):")

scaler_minmax = MinMaxScaler()
df_minmax = df.copy()
df_minmax[df.columns] = scaler_minmax.fit_transform(df[df.columns])

print("После нормализации (min=0, max=1):")
print(df_minmax.describe())

# 4. Визуализация
print("\n4. ВИЗУАЛИЗАЦИЯ РАСПРЕДЕЛЕНИЙ MedInc:")

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

axes[0].hist(df["MedInc"], bins=50, color='blue', alpha=0.7, edgecolor='black')
axes[0].set_title("MedInc (оригинальные)")
axes[0].grid(True)

axes[1].hist(df_std["MedInc"], bins=50, color='green', alpha=0.7, edgecolor='black')
axes[1].set_title("MedInc (StandardScaler)")
axes[1].grid(True)

axes[2].hist(df_minmax["MedInc"], bins=50, color='red', alpha=0.7, edgecolor='black')
axes[2].set_title("MedInc (MinMaxScaler)")
axes[2].grid(True)

plt.tight_layout()
plt.show()

print("\nЗАДАНИЕ 3 ВЫПОЛНЕНО!")
```

Рисунок 20

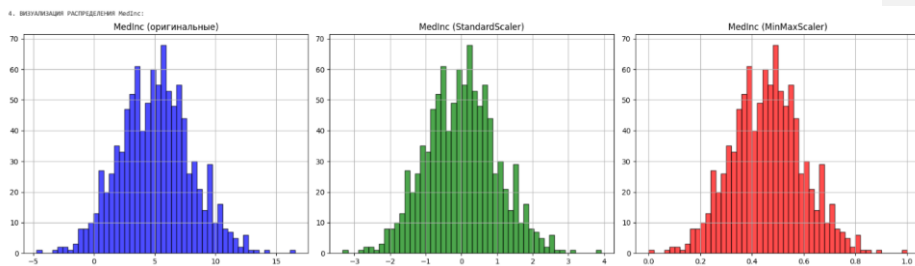


Рисунок 21

ЗАДАНИЕ 4: Кодирование категориальных признаков (Adult)

```
import pandas as pd
import numpy as np

print("-"*100)
print("ЗАДАНИЕ 4: КОДИРОВАНИЕ КАТЕГОРИАЛЬНЫХ ПРИЗНАКОВ")
print("DATASET: ADULT (OFFLINE)")
print("-"*100)

# 1. СОЗДАНИЕ ДАННЫХ
print("\n1. СОЗДАНИЕ ДАННЫХ:")

np.random.seed(42)
n = 100

df = pd.DataFrame({
    "education": np.random.choice(["Preschool", "HS-grad", "Some-college", "Bachelors", "Masters", "Doctorate"], n),
    "marital-status": np.random.choice(["Never-married", "Married", "Divorced", "Widowed"], n),
    "occupation": np.random.choice(["Tech", "Sales", "Admin", "Service", "Healthcare"], n),
    "income": np.random.choice(["<50K", ">50K"], n)
})

print("Первые 5 строк:")
print(df.head())
print(f"Размер: {df.shape}")

# 2. LABEL ENCODING (education - one-hot)
print("\n2. LABEL ENCODING для 'education':")

education_order = {
    "Preschool": 0,
    "HS-grad": 1,
    "Some-college": 2,
    "Bachelors": 3,
    "Masters": 4,
    "Doctorate": 5
}

df_encoded = df.copy()
df_encoded["education_encoded"] = df_encoded["education"].map(education_order)

print(df_encoded[["education", "education_encoded"]].head())

# 3. ONE-HOT ENCODING (marital-status, occupation)
print("\n3. ONE-HOT ENCODING для 'marital-status' и 'occupation':")

df_encoded = pd.get_dummies(df_encoded, columns=["marital-status", "occupation"], drop_first=True)

print(f"Размер после кодирования: {df_encoded.shape}")
print("\nПервые 5 строк после кодирования:")
print(df_encoded.head())

print("\nЗАДАНИЕ 4 ЗАВЕРШЕНО!")
```

Рисунок 22

ЗАДАНИЕ 4: КОДИРОВАНИЕ КАТЕГОРИАЛЬНЫХ ПРИЗНАКОВ
DATASET: ADULT (OFFLINE)

=====

1. СОЗДАНИЕ ДАННЫХ:

Первые 5 строк:

	education	marital-status	occupation	income
0	Bachelors	Divorced	Service	>50K
1	Masters	Never-married	Service	>50K
2	Some-college	Never-married	Admin	<=50K
3	Masters	Never-married	Sales	>50K
4	Masters	Divorced	Service	>50K

Размер: (100, 4)

2. LABEL ENCODING для 'education':

	education	education_encoded
0	Bachelors	3
1	Masters	4
2	Some-college	2
3	Masters	4
4	Masters	4

3. ONE-HOT ENCODING для 'marital-status' и 'occupation':

Размер после кодирования: (100, 10)

Первые 5 строк после кодирования:

	education	income	education_encoded	marital-status_Married	\
0	Bachelors	>50K	3		False
1	Masters	>50K	4		False
2	Some-college	<=50K	2		False
3	Masters	>50K	4		False
4	Masters	>50K	4		False

	marital-status_Never-married	marital-status_Widowed	\
0	False	False	
1	True	False	
2	True	False	
3	True	False	
4	False	False	

	occupation_Healthcare	occupation_Sales	occupation_Service	\
0	False	False	True	
1	False	False	True	
2	False	False	False	
3	False	True	False	
4	False	False	True	

	occupation_Tech
0	False
1	False
2	False
3	False
4	False

Рисунок 23

ЗАДАНИЕ 5: Комплексный EDA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statistics as stats
from sklearn.preprocessing import StandardScaler

print("\n*40")
print("ЗАДАНИЕ 5: КОМПЛЕКСНЫЙ EDA")
print("ИМЯ ФАЙЛА: HEART_OLIMPIAD_OFLINE")
print("\n*40")

# 1. СОЗДАНИЕ ДАННЫХ
print("\n1. СОЗДАНИЕ ДАННЫХ:")
n = 1000

df = pd.DataFrame({
    "age": np.random.randint(20, 80, n),
    "sex": np.random.choice(["Male", "Female"], n),
    "chestpain": np.random.choice(["typical", "atypical", "non-anginal"], n),
    "restingbp": np.random.randint(90, 200, n),
    "cholesterol": np.random.randint(120, 400, n),
    "maxbp": np.random.randint(70, 220, n),
    "maxrestingbp": np.random.choice(["yes", "no"], n),
    "restingecg": np.random.choice(["Normal", "ST-T", "LW"], n),
    "target": np.random.randint(0, 2, n)
})

# Проверка структуры
df.info()
df.describe()

# 2. СТРУКТУРА
print("\n2. СТРУКТУРА ДАННЫХ:")
df.info()

# 3. ПРОБНОЕ
print("\n3. ПРОБНОЕ:")
print(df.head())

# 4. ВИСУАЛИЗАЦИЯ ПРОБНОЕ
print("\n4. ВИСУАЛИЗАЦИЯ ПРОБНОЕ:")

plt.figure(figsize=(10, 6))
sns.pairplot(df)
plt.show()

# 5. ОБРАБОТКА ПРОБНОЕ
print("\n5. ОБРАБОТКА ПРОБНОЕ:")

df_clean = df.copy()
df_clean["age"] = df_clean["age"].fillna(df_clean["age"].median())
df_clean["cholesterol"] = df_clean["cholesterol"].fillna(df_clean["cholesterol"].median())
print("\n age: значение NaN")
print("\n cholesterol: значение NaN")

# 6. ОБРАБОТКА
print("\n6. ОБРАБОТКА:")

numeric_cols = ["age", "cholesterol", "restingbp", "maxbp"]

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

for i, col in enumerate(numeric_cols):
    sns.boxplot(df_clean[col], ax=axes[i])
    axes[i].set_title(f"Boxplot: {col}")
    axes[i].grid(True)

plt.tight_layout()
plt.show()

# 7. ВИСУАЛИЗАЦИЯ
def remove_outliers(df, data, col):
    Q1 = data[col].quantile(0.25)
    Q3 = data[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    return data[(data[col] >= lower) & (data[col] <= upper)]
```

Рисунок 24

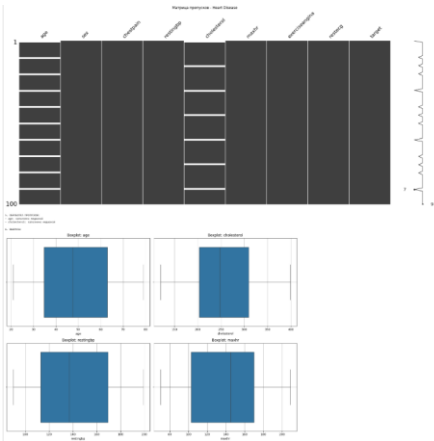


Рисунок 25

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import missingno as mso
from sklearn.preprocessing import StandardScaler

print("-"*60)
print("ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ")
print("ДАТАСЕТ: [НАЗВАНИЕ ВАШЕГО ДАТАСЕТА]")
print("-"*60)

# 1. ЗАГРУЗКА ДАННЫХ
print("\n1. ЗАГРУЗКА ДАННЫХ:")

# Подставьте свой путь к файлу
# df = pd.read_csv("путь/к/вашему/файлу.csv")
# Или создайте свой датасет

# Пример создания своего датасета
np.random.seed(42)
n = 100

df = pd.DataFrame({
    "age": np.random.randint(18, 70, n),
    "gender": np.random.choice(["Male", "Female"], n),
    "salary": np.random.randint(30000, 100000, n),
    "experience": np.random.randint(0, 30, n),
    "education": np.random.choice(["High School", "Bachelor", "Master", "PhD"], n),
    "city": np.random.choice(["Moscow", "SPb", "Kazan", "Novosibirsk"], n),
    "job_title": np.random.choice(["Engineer", "Manager", "Analyst", "Developer"], n),
    "remote": np.random.choice(["Yes", "No"], n),
    "satisfaction": np.random.randint(1, 10, n),
    "target": np.random.choice([0, 1], n) # Пример: уволился или нет
})

print("Первые 5 строк:")
print(df.head())
print(f"Размер датасета: {df.shape}")
print(f"Признаки: {df.columns.tolist()}")

# 2-8. ВЫПОЛНИТЕ АНАЛОГИЧНЫЙ АНАЛИЗ (как в предыдущих примерах)

print("\nИНДИВИДУАЛЬНОЕ ЗАДАНИЕ ВЫПОЛНЕНО!")

=====
ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ
ДАТАСЕТ: [НАЗВАНИЕ ВАШЕГО ДАТАСЕТА]
=====

1. ЗАГРУЗКА ДАННЫХ:
Первые 5 строк:
  age  gender  salary  experience  education  city  job_title  remote \
0   56   Male   55939         2   Bachelor   Kazan   Engineer   Yes
1   69   Male   78925        24   Bachelor   Kazan   Developer   No
2   46   Male   72941        18  High School   Moscow   Analyst   Yes
3   32   Male   53834        17    PhD      Novosibirsk   Engineer   Yes
4   60   Male   48047        13  High School   Moscow   Engineer   Yes

  satisfaction  target
0             9       0
1             1       1
2             9       1
3             8       0
4             1       1

Размер датасета: (100, 10)
Признаки: ['age', 'gender', 'salary', 'experience', 'education', 'city', 'job_title', 'remote', 'satisfaction', 'target']

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ ВЫПОЛНЕНО!
```

Рисунок 26

eda-pandas-lab / Lab 6 /

Add file +

...

Рассказы

Add files via upload

ab755ac · now

History

Name	Last commit message	Last commit date
..		
doc	Add files via upload	now
Задание	Add files via upload	now
Примеры	Add files via upload	now
readme.txt	Add files via upload	now

readme.txt

Лабораторная работа: Исследовательский анализ данных (ИДА)

Описание

Наш лабораторный проект изучает основные методы исследовательского анализа данных (ИДА):

- Обнаружение и обработка пропущенных значений

- Обнаружение и удаление выбросов (IQR)

- Нормализация числовых признаков (StandardScaler,MinMaxScaler,RobustScaler)

- Кодирование категориальных признаков (Label Encoding, One-hot Encoding, Target Encoding)

Выполнено

Студент: Пашков Игорь Игоревич

Рисунок 27

Вывод

В ходе выполнения лабораторной работы были изучены основные методы исследовательского анализа данных (EDA): обнаружение и обработка пропусков, выявление и удаление выбросов, масштабирование числовых признаков и кодирование категориальных переменных. Основные инструменты (pandas, missingno, scikit-learn) применены на практике при решении пяти заданий и индивидуального варианта. Датасеты приведены к модельно-готовому виду: устранены пропуски и выбросы, признаки стандартизированы и закодированы. Полученные навыки являются основой для подготовки данных в задачах машинного обучения.

ОТВЕТЫ НА ВОПРОСЫ

1. Какие типы проблем могут возникнуть из-за пропущенных значений в данных?

Пропуски могут исказить статистику, приводить к смещению оценок, снижать мощность моделей и вызывать ошибки при вычислениях.

2. Как с помощью методов pandas определить наличие пропущенных значений?

Используйте `.isna()` или `.isnull()` — они возвращают True для NaN и False для заполненных значений.

3. Что делает метод `.dropna()` и какие параметры он принимает?

Удаляет строки или столбцы с NaN. Параметры: `axis` (0 - строки, 1 - столбцы), `how` ("any" или "all"), `thresh` (минимальное количество не-NaN), `subset` (столбцы для проверки).

4. Чем различаются подходы заполнения пропусков средним, медианой и модой?

Среднее (mean) — чувствительно к выбросам.

Медиана (median) — устойчива к выбросам.

Мода (mode) — для категориальных данных.

5. Как работает метод .fillna(method='ffill') и в каких случаях он применим?

Заполняет NaN предыдущим значением (forward fill). Применим для временных рядов и последовательных данных.

6. Какую задачу решает метод .interpolate() и чем он отличается от .fillna()?

Восстанавливает пропуски на основе соседних значений (интерполяция). В отличие от .fillna(), учитывает тренд данных.

7. Что такое выбросы и почему они могут искажать результаты анализа?

Выбросы — аномальные значения, сильно отличающиеся от основной массы. Искажают среднее, дисперсию и могут приводить к переобучению моделей.

8. В чём суть метода межквартильного размаха (IQR) и как он используется для обнаружения выбросов?

$IQR = Q3 - Q1$. Выбросы — значения за пределами $[Q1 - 1.5IQR, Q3 + 1.5IQR]$.

9. Как вычислить границы IQR и применить их в фильтрации?

$lower = Q1 - 1.5 * IQR$

$upper = Q3 + 1.5 * IQR$

$df[(df[col] >= lower) \& (df[col] <= upper)]$

10. Что делает метод .clip() и как его можно использовать для обработки выбросов?

Ограничивает значения заданными границами. Заменяет значения ниже/выше границ на сами границы, сохраняя строки.

11. Зачем может потребоваться логарифмическое преобразование числовых признаков?

Чтобы "сжать" большие значения, уменьшить влияние выбросов и сделать распределение более нормальным.

12. Какие графические методы позволяют обнаружить выбросы?

Boxplot, Histograma, Scatterplot.

13. Почему важно быть осторожным при удалении выбросов из обучающих данных?

Выбросы могут быть не ошибками, а важными редкими случаями. Их удаление может потерять ценную информацию.

14. Зачем необходимо масштабирование признаков перед обучением моделей?

Чтобы признаки с большим масштабом не доминировали над другими, особенно в моделях, основанных на расстояниях и градиентных методах.

15. Чем отличается стандартизация от нормализации?

Стандартизация $\rightarrow$ среднее=0, std=1: $(x - \mu) / \sigma$

Нормализация $\rightarrow$ диапазон [0,1]: $(x - \min) / (\max - \min)$

16. Что делает StandardScaler и как рассчитываются преобразованные значения?

Преобразует значения по формуле $(x - \text{mean}) / \text{std}$. Результат имеет среднее 0 и стандартное отклонение 1.

17. Как работает MinMaxScaler и когда его использование предпочтительно?

Преобразует значения в диапазон [0,1] по формуле $(x - \min) / (\max - \min)$. Предпочтителен, когда нужно сохранить относительные различия.

18. В чём преимущества RobustScaler при наличии выбросов?

Использует медиану и IQR вместо среднего и std, поэтому устойчив к выбросам.

19. Как реализовать стандартизацию с помощью .mean() и .std() вручную в pandas?

```
df["col"] = (df["col"] - df["col"].mean()) / df["col"].std()
```

20. Какие типы моделей наиболее чувствительны к масштабу признаков?

KNN, SVM, линейная регрессия, логистическая регрессия, нейронные сети. Деревья решений не чувствительны.

21. Почему необходимо преобразовывать категориальные признаки перед обучением модели?

Модели работают только с числами. Категории не имеют арифметического смысла и должны быть закодированы в числовой формат.

22. Что такое порядковый признак? Приведите пример.

Resposta: Категории с логическим порядком. Пример: образование (начальное < среднее < высшее).

23. Что такое номинальный признак? Приведите пример.

Resposta: Категории без порядка. Пример: город (Москва, СПб, Казань).

24. Как работает метод .factorize() и для каких случаев он подходит?

Присваивает числа категориям в порядке их появления. Подходит для быстрого кодирования, но не позволяет задать порядок.

25. Как применить метод `.map()` для кодирования категориальных признаков с известным порядком?

```
mapping = {"кат1": 0, "кат2": 1, "кат3": 2}
df["col"] = df["col"].map(mapping)
```

26. Что делает класс `OrdinalEncoder` из `scikit-learn`?

Кодирует порядковые признаки, позволяя задать порядок категорий через параметр `categories`. Поддерживает работу в Pipeline.

27. В чём суть `one-hot` кодирования и когда оно применяется?

Создаёт бинарные столбцы для каждой категории. Применяется для номинальных признаков (без порядка)

28. Как избежать дамми-ловушки при `one-hot` кодировании?

Использовать `drop_first=True` в `pd.get_dummies()` или `drop='first'` в `OneHotEncoder`.

29. Как работает `OneHotEncoder` из `scikit-learn` и чем он отличается от `pd.get_dummies()`?

`OneHotEncoder` интегрируется в Pipeline, поддерживает кросс-валидацию. `pd.get_dummies()` — простой, но не поддерживает Pipeline.

30. В чём суть метода `target encoding` и какие риски он в себе несёт?

Заменяет категорию на среднее значение целевой переменной. Риски: `data leakage` (утечка данных) и `overfitting` (переобучение).